



Phage
SECURITY

SECURITY REVIEW · PREPARED FOR

Robin Markets

A Polygon yield vault for
Polymarket outcome tokens

DATE

02.06.2026

CONDUCTED BY

Pyro, Lead Security Researcher
MarketerKA, Security Researcher
YanecaB, Security Researcher
Bobai23, Security Researcher

COMMIT

47d6b4a

REMEDIATION

663332f

01 ABOUT

Phage Security

Phage Security is a team of highly skilled smart-contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike.

With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

PREVIOUS WORK

github.com/phage-security

REACH OUT

Telegram · @Pyro3b

WHAT AN AUDIT IS

An audit is a time, resource, and expertise-bound effort in which trained researchers evaluate smart contracts using a combination of manual and automated techniques to identify as many vulnerabilities as possible.

An audit can reveal the presence of vulnerabilities, but it cannot guarantee their absence. This report is a point-in-time assessment of the code at the commits named on the cover. It is not a warranty that the system is secure.

02 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	ACKNOWLEDGED
11	10	1

SEVERITY DISTRIBUTION

2 High	5 Medium	4 Low
--------	----------	-------

CLIENT	REPOSITORY	METHODS
Robin Markets	github.com/robin-markets/staking-vault-contracts	Manual review

CONTRACTS IN SCOPE

src/RobinLens.sol	src/RobinStakingVault.sol
src/RobinStakingVaultExtension.sol	src/RobinTimeLockController.sol
src/RobinTwapOracle.sol	src/libraries/AccountingLib.sol
src/libraries/IndexCalcLib.sol	src/libraries/PolymarketLib.sol
src/libraries/ShareMath.sol	src/libraries/StorageLib.sol
src/libraries/TwapMath.sol	src/libraries/VaultLib.sol
src/mixins/AccountingMixin.sol	src/mixins/PausableMixin.sol
src/mixins/PolymarketMixin.sol	src/mixins/SignaturesMixin.sol
src/mixins/YieldStrategyMixin.sol	

TIMELINE · JUNE 2026

Audit · 5 days		Remediation · 3 days	
KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
18.05.2026	22.05.2026	25.05.2026	27.05.2026

STARTING COMMIT HASH	47d6b4a93d14c91c643e4032993ecafffa7e4b7e2
FINAL COMMIT HASH	663332f3c07a56fcccb75e81e0fe2ac0b917bb16

03 PROJECT SUMMARY

What Robin Markets is

Robin lets users deposit Polymarket binary outcome tokens into a single vault and earn yield on the matched principal.

Each deposit pairs YES with NO 1:1, merges the pair into USDC.e through Polymarket's Conditional Tokens Framework, and routes it to external ERC-4626 vaults. Principal returns as outcome tokens, reduced on loss, and yield accrues in USDC.e less a basis-point protocol cut.

SECURITY POSTURE

BEFORE REMEDIATION

47d6b4a

Withdrawal auth and operator path

H-01 · M-02 · M-03

`POLY_GNOSIS_SAFE` auth never read the Safe's current owner set, so a leaked deploy-time key kept permanent authority to drain yield. Nonce cancels were miskeyed so proxy and Safe cancels no-opped, and the operator path forwarded unbounded gas.

Cross-market accounting

H-02 · M-04 · L-01 · L-02

The permissionless `supplyIdleToVaults` was reentrant through the ERC-1155 hook and could drain in-flight yield from the fee reserve. Push-deposit accounting double-counted its balance until deposits froze, loss was double-debited when one side was empty, and `nonZeroLength` went unverified.

TWAP-oracle coherence

M-05 · L-03

`submitTwap` was permissionless on non-required markets and could pin `lastTwapUpdate` to block finalization forever. `setTwapOracle` swapped the oracle pointer with no state migration, bricking every initialized market.

External vault resilience

M-01 · L-04

Bare `IERC4626` calls across `VaultLib` let one misbehaving vault DoS deposits, withdrawals, fee harvest, and the only recovery path.

`supplyIdleToVaults` also reported its unsupplied remainder as the supplied amount.

AFTER REMEDIATION

663332f

Ten of eleven findings fixed and verified. Both High findings are closed: the `POLY_GNOSIS_SAFE` branch now requires `ISafe(user).getThreshold() == 1` && `ISafe(user).isOwner(signer)`, so a signed withdrawal is only valid for a current single-owner Safe, and `supplyIdleToVaults` was made `nonReentrant`, closing the 1155-hook reentrancy into the fee reserve. The M-01 fix wraps the external ERC-4626 calls in try/catch, covering `previewRedeem`, `maxDeposit`, `maxWithdraw`, and `withdraw`, so a misbehaving external vault can no longer block deposits, withdrawals, or recovery.

The one acknowledged item, M-02, is the operator-paid gas-burn path, an asymmetric grieving vector that drains the operator hot wallet at zero attacker cost without touching user funds. The audit found a system with sound foundations. Share math, UUPS upgrade gating, and the dual-index loss/yield model held up, with the deepest issues sitting in the signed-withdrawal authorization model and the seams where multi-market accounting meets the Polymarket and ERC-4626 boundaries. The protocol is deployable once that path is bounded on-chain.

04 RECOMMENDATIONS

Beyond this audit

Four measures that extend the protection of this review after launch, independent of any individual finding.

R-01	Bug bounty	Stand up a public bounty on Immunefi or HackenProof with the top tier covering fund loss.
R-02	Invariant monitoring	Run an off-chain watcher that reconciles vault state against protocol invariants and pages on divergence. Watch that vault USDC never drops below <code>accumulatedProtocolFees</code> , that <code>maximumAdditionalMatchedTokens</code> tracks the real unpaired-token backing, and that every registered ERC-4626 vault still answers <code>previewRedeem</code> and <code>maxWithdraw</code> .
R-03	Signed-withdrawal hardening	Migrate <code>POLY_GNOSIS_SAFE</code> authorization to EIP-1271 so it honors the Safe's current owner set, and bound the gas forwarded on the operator-paid path so a hostile recipient cannot drain the operator hot wallet.
R-04	Key hygiene	The <code>TIMELOCKED_ROLE</code> controller and the owner multisig are the highest-value keys in the system. Keep them and the operator EOA on hardware signers behind a multisig with a documented rotation and incident runbook.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
<code>src/RobinStakingVault.sol</code>	Main UUPS singleton handling batch deposits, withdrawals, and signed withdrawals, upgradeable behind <code>TIMELOCKED_ROLE</code> .
<code>src/RobinStakingVaultExtension.sol</code>	Admin surface reached through the vault fallback by <code>DELEGATECALL</code> , split across the timelocked, manager, operator, pauser, and fee-harvester roles.
<code>src/RobinTwapOracle.sol</code>	Standalone UUPS oracle ingesting signed batch TWAP submissions and storing per-market accumulators and finalization data.
<code>src/RobinLens.sol</code>	Stateless aggregator with an <code>immutable vault</code> reference, providing batch view helpers and <code>checkBatchDepositCapacity</code> .
<code>src/RobinTimeLockController.sol</code>	<code>OpenZeppelin TimeLockController</code> wrapper with <code>address(0)</code> as admin, so no separate timelock admin exists.

Polygon, Solidity 0.8.31. `RobinStakingVault` and `RobinTwapOracle` are UUPS upgradeable. `RobinLens` and `RobinTimeLockController` are not.

PRIVILEGED ROLES

DEFAULT_ADMIN_ROLE. Grants and revokes every non-timelocked role, held by the owner multisig.

DEFAULT_MANAGER_ROLE. Sets the vault cap and active flag, removes and reorders vaults, and manages the Polymarket oracle list, the URI, and `setTwapSigner`, held by the owner multisig.

OPERATOR_ROLE. Controls emergency mode, per-vault emergency, `setTwapGracePeriod` up to `MAX_TWAP_GRACE_PERIOD = 240`, and oracle TWAP-required flags, held by the multisig plus a manager EOA.

PAUSER_ROLE. Sets the vault pause flags and pauses or unpauses the oracle, held by the owner multisig.

FEE_HARVESTER_ROLE. Calls `harvestProtocolFee(to)`, held by a dedicated harvester address.

TIMELOCKED_ROLE. Self-administered, gates `addVault`, the fee and onramp setters, `setExtensionAddress`, and `_authorizeUpgrade`, held exclusively by `RobinTimeLockController`.

VAULT_ROLE. Calls `initializeMarket` on the oracle when the vault first sees a new market, held by `RobinStakingVault`.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
H-01	<code>POLY_GNOSIS_SAFE</code> signed withdrawal auth does not validate the current Safe owner	HIGH	● FIXED
H-02	<code>supplyIdleToVaults</code> can be reentered through the 1155 hook to drain the protocol fee reserve	HIGH	● FIXED
M-01	A misbehaving external vault can DoS deposits, withdrawals, and recovery	MEDIUM	● FIXED
M-02	Operator pays unbounded gas when a signed-withdrawal recipient burns gas	MEDIUM	● ACKNOWLEDGED
M-03	<code>invalidateNonces</code> cannot cancel signed withdrawals for proxy or Safe users	MEDIUM	● FIXED
M-04	Push-deposit inflates <code>maximumAdditionalMatchedTokens</code>	MEDIUM	● FIXED
M-05	Non-TWAP markets can be permanently blocked from finalization	MEDIUM	● FIXED
L-01	Pool loss is counted twice when one market side is empty	LOW	● FIXED
L-02	<code>nonZeroLength</code> is user-supplied and never validated against the actual non-zero count	LOW	● FIXED
L-03	<code>setTwapOracle</code> does not migrate per-market state and bricks every initialized market	LOW	● FIXED
L-04	Incorrect return value in <code>supplyIdleToVaults</code>	LOW	● FIXED

● HIGH SEVERITY · 2 FINDINGS

H-01

HIGH

FIXED

POLY_GNOSIS_SAFE signed withdrawal auth does not validate the current Safe owner

For `POLY_GNOSIS_SAFE`, Robin authorizes a signed withdrawal with a single check, `withdrawal.user == getAddress(signer)`. That helper is a pure `CREATE2` derivation from `keccak256(signer)` and never reads the Safe's current owner set. Robin keeps trusting the deploy-time EOA forever, even after the user removes it from the Safe with `swapOwner`. A leaked or burner deployer key stays a permanent withdrawal authority, and the attacker drains USDC yield to their own address by setting `yieldRecipient` in the signed message.

The same check exists in Polymarket's CTF Exchange, where it is safe because a CTF order is an atomic trade with proceeds hardcoded back to `order.maker`. Robin reuses the check but adds an attacker-controlled `yieldRecipient`, so authorization that meant "permission to trade" becomes "permission to send USDC anywhere".

```
} else if (signatureType == DataTypes.SignatureType.POLY_GNOSIS_SAFE) {
    return user == $.polymarketFactoryHelper.getAddress(signer);
}
```

`getAddress` is `CREATE2(salt = keccak256(signer))`, with no call into the Safe, no `isOwner`, no `getThreshold`, no EIP-1271. The nonce bitmap is keyed on `user`, not `signer`, so the user cannot retire one signing key without consuming every nonce on the Safe.

1. Bob deployed his Safe with `0xOLD`, then rotated to a hardware wallet via `swapOwner(SENTINEL, 0xOLD, 0xNEW)`, so `isOwner(0xOLD)` is now false.
2. `0xOLD` later leaks. The attacker signs a withdrawal with `signer = 0xOLD`, `user = getAddress(0xOLD)`, `yieldRecipient = attacker`, and `minYesTokens = minNoTokens = 0`.
3. Robin checks `getAddress(0xOLD) == user` and passes. Outcome tokens land in the Safe, USDC yield goes to the attacker, and Bob's position is force-exited at any price.

IMPACT

A leaked or rotated-out deploy-time key keeps permanent authority to divert every yield epoch to an attacker address and force-exit the position at a zero floor. The nonce bitmap is keyed on the Safe, so there is no per-signer revocation, and the current owner cannot even sign for the Safe because `getAddress(0xNEW)` is a different address. The bug inverts the Safe ownership model.

RECOMMENDATION

Replace the static derivation with EIP-1271 delegation against the Safe, which honors the Safe's current owner set and threshold. This is the path Polymarket itself took with `POLY_1271` in its V2 upgrade.

```
    } else if (signatureType == DataTypes.SignatureType.POLY_GNOSIS_SAFE) {  
-    return user == $.polymarketFactoryHelper.getSafeAddress(signer);  
+    return IERC1271(user).isValidSignature(digest, withdrawal.signature) == 0x1626ba7e;  
    }
```

The minimum patch is `ISafe(user).isOwner(signer)`, but it does not close the multi-sig variant. EIP-1271 closes both rotation and multi-sig in one change.

H-02

HIGH

FIXED

supplyIdleToVaults can be reentered through the 1155 hook to drain the protocol fee reserve

`_batchWithdraw` transfers outcome tokens to the user before paying the USDC yield:

```
_giveOutcomeTokens(vars.ids, vars.amts, user);  
(uint256 yield, uint256 protocolFee) = _handleYieldPayout(user, yieldRecipient,  
vars.totalYield, wrapYieldToPolyUsd);
```

`_giveOutcomeTokens` calls the CTF's `safeBatchTransferFrom`, which invokes the recipient's `onERC1155BatchReceived` hook after the balance update. At that point the vault holds the in-flight yield as idle USDC alongside the `accumulatedProtocolFees` reserve, because `_ensureUsdcBalance` already pulled `splitNeeded + yield` from Yearn and `_split` consumed only `splitNeeded`.

`supplyIdleToVaults` is permissionless and reachable through the vault fallback:

```
function supplyIdleToVaults() external returns (uint256 supplied) {  
    return _trySupplyIdleToVaults();  
}
```

Inside the hook the attacker calls `supplyIdleToVaults`. `_trySupplyIdleToVaults` computes `idle = balance - accumulatedProtocolFees = yield` and sweeps that yield back into Yearn, dropping the vault balance to `accumulatedProtocolFees`. Control returns to `_handleYieldPayout`, which pays `yield` to the user out of what is now the fee reserve and then bumps `accumulatedProtocolFees += protocolFee`.

IMPACT

The user is paid the correct amount, but the cash comes from the protocol fee reserve while the real yield sits in Yearn as untracked principal. Repeated withdrawals drain the fee reserve and corrupt internal accounting.

RECOMMENDATION

Add `nonReentrant` to `supplyIdleToVaults`.

```
function supplyIdleToVaults() external returns (uint256 supplied) {  
+ function supplyIdleToVaults() external nonReentrant returns (uint256 supplied) {  
    return _trySupplyIdleToVaults();  
}
```

● MEDIUM SEVERITY · 5 FINDINGS

M-01

MEDIUM

FIXED

A misbehaving external vault can DoS deposits, withdrawals, and recovery

The vault integrates registered ERC-4626 vaults through `VaultLib`. Most external calls into them are bare, with no try/catch, so a single registered vault that reverts on any of these calls bricks the user-facing path that reaches it.

- `_getVaultValue ( balanceOf , previewRedeem )` runs as the first external operation in `batchDeposit` and `batchWithdraw` through `_getTotalVaultValue`, which iterates every registered vault. One reverting `previewRedeem` freezes all TVL.
- `_withdrawFromVaults ( withdraw )` has no flag skip, so an `emergencyActivated` vault that still reports `maxWithdraw > 0` and reverts on `withdraw` DoSes every withdrawal path and `harvestProtocolFee`.
- `_getAvailableCapacity`, `_withdrawAllFromVault`, and `_withdrawMaxFromVault` are all bare.
- `enableVaultEmergency` and `enableEmergencyMode` call `_withdrawMaxFromVault` before setting the emergency flag, so a reverting vault leaves the flag unset.

Recovery falls back to the manager-gated `removeVault`, which itself calls the bare `_withdrawAllFromVault`, leaving a UUPS upgrade as the only escape.

```
function _getVaultValue(address vault) private view returns (uint256) {
    uint256 shares = IERC4626(vault).balanceOf(address(this));
    return IERC4626(vault).previewRedeem(shares);
}
```

IMPACT

One misbehaving or compromised registered vault can DoS `batchDeposit`, `batchWithdraw`, `harvestProtocolFee`, and the emergency recovery path, freezing all deposits and withdrawals until a UUPS upgrade.

RECOMMENDATION

Wrap every external ERC-4626 call in `VaultLib` in try/catch and skip flagged vaults.

```
uint256 shares = IERC4626(v.vault).withdraw(toWithdraw, address(this),
address(this));
emit IRobinStakingVaultEvents.VaultWithdrawal(v.vault, shares, toWithdraw);
remaining -= toWithdraw;
+   try IERC4626(v.vault).withdraw(toWithdraw, address(this), address(this)) returns
+   (uint256 shares) {
+       emit IRobinStakingVaultEvents.VaultWithdrawal(v.vault, shares, toWithdraw);
+       remaining -= toWithdraw;
+   } catch {
+       continue;
+   }
```

M-02

MEDIUM

ACKNOWLEDGED

Operator pays unbounded gas when a signed-withdrawal recipient burns gas

`executeSignedWithdrawal` is operator-paid. It ends with `ctf.safeBatchTransferFrom(vault, withdrawal.user, ...)`, which invokes `withdrawal.user.onERC1155BatchReceived` with all forwarded gas, no stipend and no try/catch. An attacker can deploy code at `withdrawal.user` whose hook returns the correct selector but burns roughly 5M gas first, draining the operator's hot wallet at zero cost to itself.

```
_giveOutcomeTokens(vars.ids, vars.amts, user); // forwards all gas to
user.onERC1155BatchReceived
(uint256 yield, ) = _handleYieldPayout(user, yieldRecipient, vars.totalYield,
wrapYieldToPolyUsd);
```

`_verifySignerForUser` ties `signer` to `user` but does not constrain the code deployed at `user`. An EOA with EIP-7702 is exploitable, since a delegated EOA runs its delegate code on the hook. `POLY_GNOSIS_SAFE` is exploitable, since the attacker deploys their own Safe and calls `setFallbackHandler` to point at a gas-burning handler. `POLY_PROXY` is not exploitable, since its `onERC1155BatchReceived` is a pure selector return baked into the clone bytecode.

IMPACT

At 50 gwei on Polygon the operator pays roughly 0.10 USD per malicious withdrawal. One funded EOA can pre-sign thousands of nonces and drain the operator hot wallet, taking the limit-order backend offline. Attacker cost per attack is zero, and USDC yield still lands after the burn, so the attacker also collects their normal payout.

RECOMMENDATION

Switch the operator-paid path to a pull pattern: credit `claimableOutcomeTokens[user][id]` and let users claim on their own gas. If a pull pattern is too invasive, cap the gas forwarded to the recipient with an assembly `call` at a fixed limit such as 200K and revert on overrun. A code-length check on `user` does not help, because `setFallbackHandler` can be re-set after a clean check.

M-03

MEDIUM

FIXED

invalidateNonces cannot cancel signed withdrawals for proxy or Safe users

Cancel and execute use different keys for the same nonce bitmap. For `POLY_PROXY` and `POLY_GNOSIS_SAFE`, a user calling `invalidateNonces` from their EOA writes a different storage slot than the executor reads, so the cancel is a silent no-op and the withdrawal still executes. Raw EOA withdrawals where `user == signer` are unaffected.

```
// RobinStakingVaultExtension.sol, cancel keys by msg.sender
function invalidateNonces(uint256[] calldata nonces) external {
    _invalidateNonces(msg.sender, nonces);
    emit IRobinStakingVaultEvents.NoncesInvalidated(msg.sender, nonces);
}
```

```
// SignaturesMixin.sol, execute keys by withdrawal.user
(bool isUsed, , ) = _getNonceInformation(withdrawal.user, withdrawal.nonce);
if (isUsed) revert WithdrawalNonceUsed(...);
$.nonceBitmap[withdrawal.user][wordPos] |= bit;
```

For proxy and Safe types `withdrawal.user` is the proxy or Safe address, not the EOA, so the two keys never match. `invalidateNonceWord` has the same root cause, so a 256-nonce mass-cancel is also a no-op.

1. Bob signs `signer = 0xAAAA`, `user = 0xPROXY`, `nonce = 7`, type `POLY_PROXY`.
2. Bob cancels from MetaMask, which sets the bit on `nonceBitmap[0xAAAA]` while `nonceBitmap[0xPROXY]` stays zero. The UI shows cancelled.
3. The backend reads `nonceBitmap[0xPROXY]` as zero, so the withdrawal executes anyway.

IMPACT

Proxy and Safe are the default Polymarket flow, and cancellation is the only on-chain way to revoke a signed withdrawal before expiry. With cancel broken the user has handed the operator an irrevocable authorization for the lifetime of the order, so a market move or a compromised operator key realizes losses the user believed they had cancelled.

RECOMMENDATION

Take the wallet address and `signatureType` as parameters and reuse `_verifySignerForUser` to authorize the caller.

```
function invalidateNonces(uint256[] calldata nonces) external {  
    _invalidateNonces(msg.sender, nonces);  
    emit NoncesInvalidated(msg.sender, nonces);  
}  
+ function invalidateNonces(address user, DataTypes.SignatureType signatureType, uint256[]  
+   calldata nonces) external {  
+   if (!_verifySignerForUser(msg.sender, user, signatureType)) revert  
+   InvalidWithdrawalSignature();  
+   _invalidateNonces(user, nonces);  
+   emit NoncesInvalidated(user, nonces);  
+ }
```

M-04

MEDIUM

FIXED

Push-deposit inflates

maximumAdditionalMatchedTokens

The vault tracks a global counter `maximumAdditionalMatchedTokens`, how much USDC could enter the yield pool if all unpaired tokens were matched, and every deposit reverts if the counter exceeds vault capacity. The update logic in `_prepareDepositMintShares` was written for pull-deposits, where the vault reads its token balance before tokens arrive. For push-deposits the order is reversed: tokens land on the vault first, then the code reads the balance and adds the deposit amount on top, counting the same tokens twice.

```
(uint256 oldYes, uint256 oldNo) = getUnpairedTokens(conditionId);
_updateMaxPotential(oldYes, oldNo, oldYes + yesAmount, oldNo + noAmount);
```

On the pull path `getUnpairedTokens` returns the balance before the deposit, so `oldYes + yesAmount` is the expected new balance. On the push path it returns the balance after the deposit, so `oldYes + yesAmount` is twice the deposit. Each push-deposit that pairs with existing opposite-side tokens leaves a permanent phantom value in the counter.

IMPACT

Any user can permanently block all deposits vault-wide by repeatedly push-depositing into markets with opposite-side unpaired tokens. No funds are lost and withdrawals and yield are unaffected, but the vault stops accepting new deposits until an admin disables the capacity check.

RECOMMENDATION

Reconstruct the true pre-transfer state on the push path by subtracting the deposit amounts.

`tokensPrePushed` is already available in `_batchDepositCore`.

```
-(uint256 oldYes, uint256 oldNo) = getUnpairedTokens(conditionId);
__updateMaxPotential(oldYes, oldNo, oldYes + yesAmount, oldNo + noAmount);
+ (uint256 currentYes, uint256 currentNo) = getUnpairedTokens(conditionId);
+ if (tokensPrePushed) {
+   _updateMaxPotential(currentYes - yesAmount, currentNo - noAmount, currentYes,
+ currentNo);
+ } else {
+   _updateMaxPotential(currentYes, currentNo, currentYes + yesAmount, currentNo +
+ noAmount);
+ }
```

M-05

MEDIUM

FIXED

Non-TWAP markets can be permanently blocked from finalization

`submitTwap` is permissionless and accepts unsigned data for markets that do not require TWAP. That branch routes through `_applyDefaultTwap`, which unconditionally sets `lastTwapUpdate = block.timestamp`:

```
function _applyDefaultTwap(bytes32 conditionId) internal {
    ...
    market.lastTwapUpdate = uint40(block.timestamp);
}
```

When the backend later submits the real finalization, `_applyFinalTwap` enforces both bounds:

```
if (twapData.marketEndedAt ≤ market.lastTwapUpdate) revert TwapTimestampInvalid(...);
if (twapData.marketEndedAt > block.timestamp) revert TwapTimestampInvalid(...);
```

`marketEndedAt` is the historical Polymarket resolution timestamp. It cannot be both greater than `lastTwapUpdate`, which any user pinned to `block.timestamp`, and less than or equal to `block.timestamp`. Any user can call `submitTwap` every block to keep finalization blocked.

1. A market resolves on Polymarket at $t = R$.
2. The attacker calls `submitTwap` with `required = false`, setting `lastTwapUpdate = T` with $T > R$.
3. The backend submits finalization with `marketEndedAt = R`, which reverts `TwapTimestampInvalid`, and this repeats every block.

IMPACT

A single permissionless caller can strand a market's principal and yield by blocking finalization indefinitely. Even a one-time submission lets yield be split 50/50 after the market has resolved.

RECOMMENDATION

In `_applyFinalTwap` for non-required markets, clamp instead of revert. The default-price extrapolation already covered the gap at the correct 50:50 weighting.

```
if (twapData.marketEndedAt ≤ market.lastTwapUpdate) {
    revert TwapTimestampInvalid(market.lastTwapUpdate, twapData.marketEndedAt,
market.lastTwapUpdate + 1);
}
+     uint256 effectiveEnd = twapData.marketEndedAt > market.lastTwapUpdate
+     ? twapData.marketEndedAt : market.lastTwapUpdate;
uint256 timeToEnd = twapData.marketEndedAt - market.lastTwapUpdate;
+     uint256 timeToEnd = effectiveEnd - market.lastTwapUpdate;
```


● LOW SEVERITY · 4 FINDINGS

L-01

LOW

FIXED

Pool loss is counted twice when one market side is empty

When a market takes a loss, `IndexCalcLib.calculateIndexes` debits the same `delta` twice. The `LOSS` branch reduces both `lossIndexYes` and `lossIndexNo` by the full `delta` :

```
vars.yesDelta = vars.delta;
vars.noDelta = vars.delta;
```

Then `_applyExcessLossReduction` reduces `yieldReductionFactor` by the part of the loss past the pairable backing:

```
uint256 pairableTokens = yesTokenAssets < noTokenAssets ? yesTokenAssets : noTokenAssets;
if (delta ≤ pairableTokens) return;
uint256 excessUsdc = delta - pairableTokens;
```

Only the paired slice `min(YES, NO)` is backed by splittable pool USDC. A loss inside that slice belongs on `lossIndex` . A loss past it must hit yield instead, because the surviving side is backed by unpaired outcome tokens.

1. A market has 100 YES shares, 0 NO shares, and 10 USDC of unclaimed YES yield, so `pairableTokens = 0` .
2. The pool loses 5 USDC. The `LOSS` branch debits `lossIndexYes` by 5.
3. `_applyExcessLossReduction` sees `excessUsdc = 5` and debits `yieldReductionFactor` by 5, so YES is charged the same 5 USDC loss twice.

IMPACT

When one market side is empty, depositors on the surviving side absorb the realized loss twice, once against their outcome-token backing and once against yield. The double-count stops occurring once both sides hold pairable backing.

RECOMMENDATION

Cap the side `lossIndex` reduction at `pairableTokens` instead of applying the full `delta` .

```
    } else {  
        vars.yesDelta = -vars.delta;  
        vars.noDelta = -vars.delta;  
        uint256 yesTokenAssets = vars.totalSharesYes > 0 ?  
vars.totalSharesYes.mulDiv(result.lossIndexYes, DataTypes.INDEX_SCALE, Math.Rounding.Floor) :  
0;  
        uint256 noTokenAssets = vars.totalSharesNo > 0 ?  
vars.totalSharesNo.mulDiv(result.lossIndexNo, DataTypes.INDEX_SCALE, Math.Rounding.Floor) : 0;  
+        uint256 pairableTokens = yesTokenAssets < noTokenAssets ? yesTokenAssets :  
noTokenAssets;  
+        uint256 lossToIndex = vars.delta < pairableTokens ? vars.delta : pairableTokens;  
+        vars.yesDelta = lossToIndex;  
+        vars.noDelta = lossToIndex;
```

L-02

LOW

FIXED

`nonZeroLength` is user-supplied and never validated against the actual non-zero count

`nonZeroLength` is passed by the caller in `batchDeposit` and `batchWithdraw` but is only checked to be at least `vars.len`, never against the actual count of non-zero entries across `yesAmounts` and `noAmounts`:

```
if (nonZeroLength < vars.len) revert LengthMismatch();
```

A caller can pass any length when the real non-zero count is between `vars.len` and `vars.len * 2`, so tail entries stay zero-initialized (`ids[i] = 0`, `amts[i] = 0`) and pass into `_takeOutcomeTokens` and `_giveOutcomeTokens` unchanged.

IMPACT

On the pull-deposit path this transfers zero amounts of token id 0, which is harmless on-chain but produces malformed `TransferBatch` events that break off-chain indexers. On the push-deposit path the `keccak256` of the inflated arrays never matches `receivedHash`, so it reverts with a misleading `PushDepositMismatch` instead of `LengthMismatch`, masking the real cause.

RECOMMENDATION

Count the actual non-zero entries before array allocation and require the count equals `nonZeroLength`. Fold the per-entry zero-amount check into the same pass, and apply the pattern in both `_batchDepositCore` and `_batchWithdraw`.

```
uint256 actualNonZeroCount;
for (uint256 i = 0; i < vars.len; i++) {
    if (yesAmounts[i] == 0 && noAmounts[i] == 0) revert ZeroAmount();
    if (yesAmounts[i] > 0) actualNonZeroCount++;
    if (noAmounts[i] > 0) actualNonZeroCount++;
}
if (nonZeroLength != actualNonZeroCount) revert LengthMismatch();
```

L-03

LOW

FIXED

setTwapOracle does not migrate per-market state and bricks every initialized market

setTwapOracle is gated by TIMELOCKED_ROLE , and the underlying setter just overwrites the storage pointer:

```
function _setTwapOracle(address twapOracle_) internal {
    _getAccountingStorage().twapOracle = IRobinTwapOracle(twapOracle_);
}
```

No per-market state is copied to the new oracle. Every market initialized on the old oracle is uninitialized on the new one, while the vault's MarketState.lastYieldTwapCheckpointYes still carries the old accumulator value. After the swap, every deposit and withdraw runs _updateYieldIndexes , which calls getTwapAccumulatorYes and reverts MarketNotInitialized when the new oracle returns (0, 0) .

Recovery is also blocked. vault.initializeMarket reverts MarketAlreadyInitialized because the vault still has marketInitTimestamp > 0 . Initializing directly on the new oracle under VAULT_ROLE then hits an underflow in IndexCalcLib.calculateIndexes , where input.twapAccumulatorYes is 0 from the fresh oracle and market.lastYieldTwapCheckpointYes is the non-zero old value, so the subtraction reverts Panic(0x11) .

IMPACT

A single timelocked oracle swap DoSes every deposit and withdraw on every active market, with no in-protocol recovery path. It needs the TIMELOCKED_ROLE , so it is a privileged-operation footgun rather than a permissionless attack.

RECOMMENDATION

Remove the bare setter. Replace it with a migration function that takes the existing condition IDs, initializes each one on the new oracle, and resets the vault's per-market checkpoints in the same transaction.

L-04

LOW

FIXED

Incorrect return value in `supplyIdleToVaults`

`supplyIdleToVaults` returns a `supplied` variable intended to report how much USDC was moved. The internal call actually returns the remaining unsupplied idle USDC:

```
function supplyIdleToVaults() external returns (uint256 supplied) {  
    return _trySupplyIdleToVaults();  
}
```

```
function trySupplyIdleToVaults(uint256 reservedUsdc) external returns (uint256 remaining) {  
    uint256 idle = _getIdleUsdc(reservedUsdc);  
    if (idle == 0) return 0;  
    remaining = _trySupplyToVaults(idle, reservedUsdc);  
}
```

Passing the value straight through reports the exact opposite of reality: 0 when all funds were supplied, and the full idle balance when nothing was supplied.

IMPACT

Any caller or integrator reading the return value gets an inverted figure. No funds are at risk, but off-chain accounting and monitoring that trust the return value are wrong.

RECOMMENDATION

Compute the actual amount supplied by measuring the idle balance before the internal call and subtracting the returned `remaining`.